2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD) | 979-8-3503-5866-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICICT4SD59951.2023.10303336

Static Analysis based Malware Detection for Zero-day Attacks in Android Applications

Jabunnesa Jahan Sara
Department of Computer Science and Engineering
Military Institute of Science and Technology
Dhaka, Bangladesh
sara538t@gmail.com

Md. Shohrab Hossain
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh
shohrab29@gmail.com

***Abstract*— Android is the most popular smartphone operating system with a market share of 68.6% in Apr 2023. Hence, Android is a more tempting target for cybercriminals. This research aims at contributing to the ongoing efforts to enhance the security of Android applications and protect users from the ever-increasing sophistication of malware attacks. Zero-day attacks pose a significant challenge to traditional signature-based malware detection systems, as they exploit vulnerabilities that are unknown to all. In this context, static analysis can be an encouraging approach for detecting malware in Android applications, leveraging machine learning (ML) and deep learning (DL)-based models. In this research, we have used single feature and combination of features extracted from the static properties of mobile apps as input(s) to the ML and DL based models, enabling it to learn and differentiate between normal and malicious behavior. We have evaluated the performance of those models based on a diverse dataset (DREBIN) comprising of real-world Android applications features, including both benign and zero-day malware samples. We have achieved F1 Score 96% from the multi-view model (DL Model) in case of Zero-day malware scenario. So, this research can be helpful for mitigating the risk of unknown malware.**

***Keywords*— Android malware, Static analysis, API, OpCode, Zero-day malware, NLP**

I. INTRODUCTION

Because of the convenience and efficiency of numerous applications, as well as the growing development in the hardware and software on smart devices, mobile phones and its related applications are growing rapidly. By 2023, it is expected that 4.3 billion people will own a smartphone [1]. As a result, malware authors have a wealth of attack options, necessitating the development of robust Android malware detection systems. Although Android has a number of security features built in, there are a number of design faults and security weaknesses that have become hazards to its users. It is also necessary to be aware of these dangers in order to execute a proper malware detection and vulnerability analysis. We can define Android malware as a malware, which infects or snatch data particularly from the Android operating system-based mobile devices. Machine learning (ML) is a division of AI and depend on previous data ML algorithms make their predictions. Several steps are involved in the lifecycle of a machine learning process and those steps should be accomplished sequentially [2]. Supervised learning, unsupervised learning, semi-supervised learning, reinforcement learning, and deep learning [3] are the major subcategories of ML. In case of supervised learning, we are using a labeled dataset for training purpose.

The contributions of this paper are (i) to develop ML based classification model for android malware detection, (ii) to develop DL-based classification model, and to detect zero-day attacks for a static analysis-based approach using DL-based model.

In our paper, we propose a model that aims to detect zero-day attacks for static analysis using ML (Machine Learning) and DL (Deep Learning) algorithms. So, the two classification methods include- Machine Learning (ML) based classification and Deep Learning (DL) based classification.

Rest of the paper is organized as follows. In section II, related works are described. In section III, we present our proposed model and work Methodology along with Dataset and zero-day attack prevention scenario. In section V, we include experiment results for Random Forest, Logistic Regression, Support Vector Machine (SVM) and zero-day analysis. Finally in section VI, has the concluding remarks.

II. BACKGROUND STUDY

The foundation of the Android operating system lies in the Linux kernel. Linux, being an open-source platform, examines the proof of the course, provides networking drivers and protocols, manages virtual memory, device power, and ensures security [4]. Security is already incorporated into Android. Apps Applications are unable to access system resources unless the user provides permissions during installation or when making any changes. It's a distinct operating system for privileged users [2]. By generating suitable features set and train ML model based on that to find out malicious APKs, we have to do APK analysis. So, there are various APK analysis methodologies that include static, dynamic and hybrid analysis. The hybrid analysis combines both static and dynamic analysis. Static analysis can be performed on APKs and does not need to be executed at runtime.

A. Related Works

For extracting code level features from APK, APKTool can be used because it has the static analyzer and decompilation facilities and this approach for malware detection was proposed in [5] based on Android manifest permission analysis. The dataset utilized to train four distinct machine learning algorithms was the AndroZoo repository. The model validity process was performed using Random Forest (RF), SVM, NB, and K-Means, with RF producing the maximum accuracy for this model with 82.5 percent precision and 81.5 percent recall [5]. However, when compared to other studies in the same field, the model accuracy is not up to the mark. Popular static analysis technique is manifest-based static analysis. The permissions-based approach suggested in SigPID[6]. With over 90 percent accuracy, the support-vector machine (SVM)

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:59:56 UTC from IEEE Xplore. Restrictions apply.

outperformed compare to the other ML algorithms. JOWMDroid Framework [7] proposes a static analysis-based model. One binary based matrix was created and for calculating the initial weight they used the RF, SVM, Logistic regression (LR), and KNN ML models [7]. Correlations between features were not considered. When the connection between features is taken into account, the model's accuracy in detecting malware improves.

Deep learning may be utilized for a range of tasks because to the accessibility of large amounts of data and affordable technology. There have been several initiatives to expand the application of deep learning beyond image recognition or natural language processing to virus detection in the context of security [8]. The AdMat model (CNN) Matrix-based method and apps were categorized and handled like pictures in this paradigm [8]. The adjacency matrix for applications was generated. These matrices served as the CNN's input pictures, and the model was trained to differentiate between harmful and helpful applications [8]. Although this model has the high accuracy but performance depends on the number of characteristics used and no zero-day scenario has been considered.

B. Gap Analysis

Existing research have the following limitations:

- Some works [6], [7] did not consider correlation of features, thus it may miss crucial analysis and produces low accuracy rate for model.
- Others [7] just consider two features those are permission and service, which may lead low accuracy model.
- In paper [8], we found that accuracy is high but No consideration for zero-day attacks scenario.
- In another paper [9], they have considered just one feature which is OpCode and it requires update the training model for new malware.

So, from the existing research, we found that there are limitations of features observation, lack of Zero-day attacks scenario and less research found on features combination. In our research, we have tried to mitigate these limitations.

III. PROPOSED MODEL

Our proposed model methodology is based on two approaches. First, Machine Learning Approach and Second, Deep Learning Approach for Zero Day Attack Prevention. In our research, we focus on the development of a static analysis-based malware detection model specifically tailored for zero-day attack scenarios in Android applications. The proposed model combines the strength of ML and DL algorithms to enhance the accuracy and efficiency of malware detection.

A. Dataset

For our model, we have used Drebin dataset [10] and it is a common and widely used dataset for malware detection, according to our research. Drebin has 123,453 benign programs and 5,560 malicious applications from 179 different malware families [3], [10]. Straightly we can use DREBIN dataset in our smartphones, along with the analysis starting whenever we are going to download new applications into our device. As a result, DREBIN can protect consumers who download apps from untrustworthy sources like websites and third-party stores.

B. Machine Learning Approach

From Fig.1 we can see that we read the Drebin Dataset files (sha256 family.csv and feature vectors.zip). Then we did Feature Selection and used the Random Forest Classifier to get the best features. We selected the top 3 best features to determine malware in an android app using the Random Forest algorithm. The features are I. Permissions, II. Opcode and III. API Calls. Next, we trained the model using three different algorithms (Random Forest, Logistic Regression, and Support vector machine (SVM). Then we tuned the hyperparameters of the algorithms and trained again till we found the best results (Accuracy, F1 Score, Precision, Recall).

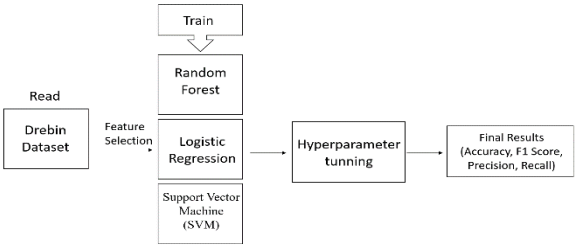


Fig. 1. ML based Proposed Model

In Fig. 1, we can see that we have used three different algorithms. Firstly, "Random Forest Classifier" is the name given to the classification method that consists of several decision trees [11]. In our experiment, we have tuned a variety of hyperparameters to obtain the best result. Like-

max features: These are the most attributes that Random Forest is permitted to test in a single tree. To assign the most features, Python offers a variety of alternatives. Such as: Auto, None and sqrt. n estimators: 10, 100, 200 and 1000.

Then, we can see, we have used "Logistic Regression" to detect the probability of a target variable. In our experiment, we have tuned a variety of hyperparameters of Logistic Regression to obtain the best result. Like- penalty: 'l1', 'l2', 'elasticnet', 'none', default='l2'. C: float, default=1.0. solver: 'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'.

Finally, we have used Support Vector Machine (SVM). "Support Vector Machine" (SVM) can resolve classification and regression issues and it is based on supervised machine learning approach [12], [13]. In our experiment, we have tuned a variety of hyperparameters of SVM to obtain the best result. Like- gamma: 'scale', 'auto' or float, default='scale' Kernel coefficient for 'rbf', 'poly' and 'sigmoid'. C: float, default=1.0

C. Zero Day Attack Prevention Using Deep Learning

A software vulnerability which is unrevealed to those who should be worried about its deadening or known but for which no patch has been prepared is called a Zero-day Attack (also known as 0-day). Hackers can use the vulnerability to harm applications, data, additional systems, or a network until it is patched. A zero-day exploit, often known as a zero-day attack, is an exploit that targets a zero-day vulnerability. By using deep neural networks, we can solve the zero-day attack scenario. Convolutional neural networks are very popular for these kinds of tasks. We

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:59:56 UTC from IEEE Xplore. Restrictions apply.

wanted to approach this whole CNN problem as an NLP (Natural Language Processing) problem.

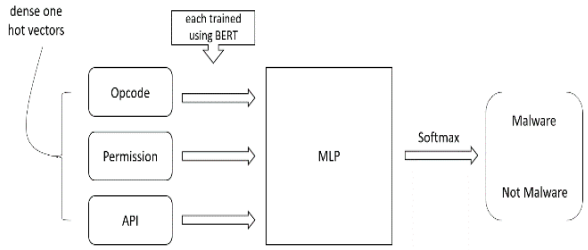


Fig. 2. DL based Proposed Model

In Fig. 2, we can find that we extracted Opcode, API and Permissions from the Drebin dataset and made dataframes. These dataframe are then converted into Dense One Hot Vectors and treat each row as a corpus. Our architecture consists of three views; first train the Opcode corpuses, then Permission following API corpuses using CNN. We modeled the CNN architecture as an NLP task and then used Google's Transformer based Masking model 'BERT' to train each of the dataframe to identify malicious patterns in an app. Then we have created a single feature representation by integrating every single feature vector (total three) together. Taking that as an input in MLP (Multilayer Perceptron) which predict the possibility of the app to be a malware or regular app. Selecting or rating features like other systems do to categorize input information, we only extract raw features. Our method's main goal is to decouple specialized knowledge in order to reduce the role of the human analyst. Utilizing APKTool every app's.dex file is broken down into .smali files [13], [14]. By utilizing APKTool we developed feature set. Using this proposed model, we can detect Zero-day Malware and it will perform better than our proposed ML model in case of Zero-day attack scenario.

IV. Experiment Results

The dataset we used in our experiment is the famous Drebin malware dataset. To use this dataset, we need to manually select features then train our model on it. To find the optimum features in our use case we used a feature selection algorithm to justify the priority of features to detect a malware. We have used 3 informative features to build a forest and compute the feature importance's, then select the top three ranked feature importance's of the forest which is based on a classification task. We used a Random Forest Classifier model for this task. With no class weights and selected criterion function as 'Gini' and an n estimator of 1000. The result is shown in the Table I.

TABLE I. Features Ranking

Feature	Rank (score)
Opcode	1 (0.271525)
Real Permissions	2 (0.146973)
API Call	3 (0.125988)
Intent	4 (0.108765)
Service	5 (0.107854)
Provider	6 (0.069560)
URL	7 (0.062323)

For result evaluation approach, we have used four standard metrics: accuracy, precision, recall rate, and F-score.

A. Using Random Forest (RF)

In Fig. 3, we can observe that by tuning hyperparameters, we are getting different results. We can notice that for n estimators: 1000 and max feature: auto, we got the best F1 score (93%) as well as accuracy. As we aware of more estimators refers that more decision trees work together, thus it helps to capture complex patterns in dataset and as well as reduces overfitting.

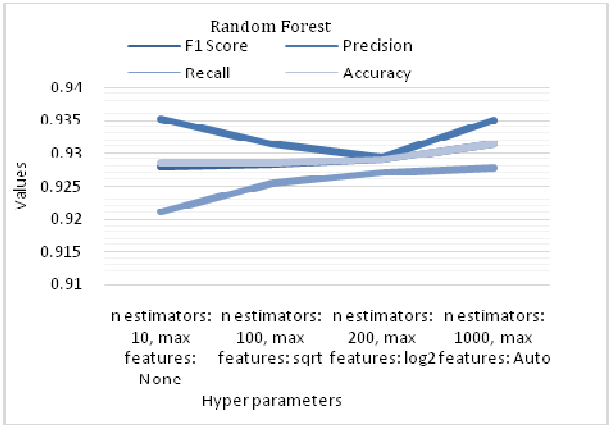


Fig. 3. Graphical Representation for Experimental Results of Random Forest

B. Using Logistic Regression (LR)

In Fig. 4, we can observe that by tuning hyperparameters, we are getting different results. We can notice that for key hyperparameter C: 0.01 we got the best F1 score 82% and best Accuracy 83%. Smaller C values help to improve F1score by controlling against overfitting problem.

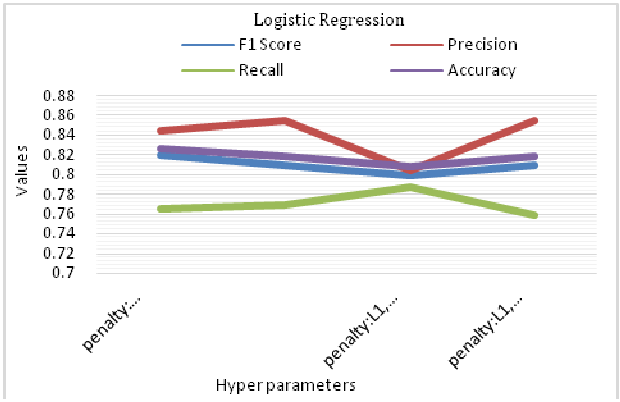


Fig. 4. Graphical Representation for Experimental Results of Logistic Regression

C. Using Support Vector Machine (SVM)

In Fig. 5, we can observe that by tuning hyperparameters, we are getting different results. We can notice that for key hyperparameters gamma: 1e-2, we got the best F1 Score 89% and Accuracy 89%. It helps to balance between capturing significant details and avoiding overfitting as a result we find the best accuracy rate.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:59:56 UTC from IEEE Xplore. Restrictions apply.

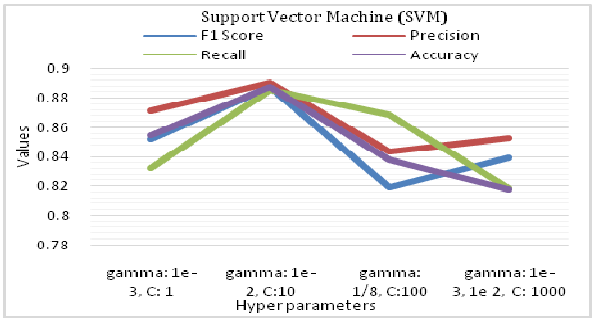


Fig. 5. Graphical Representation for Experimental Result of Support Vector Machine (SVM)

D. *Zero-Day Analysis*

Using Google's BERT Transformer model, all tested architectures utilized in the Opcodes and the APIs for CNN contain just one convolutional layer. Experiment for Opcode, Permissions and API (CNN) and other features- In our experiment, Optimizer, Learning Rate, Hidden Layers, Padding, Token Size, Batch, Activation Function, and Epoch these hyperparameters were experimentally set and applied. We have achieved F1 Score- 95% for Opcode and Permissions and 81% for API calls. In the Fig. 6, 7, 8, and 9, we can find that by tuning hyperparameters for 5 different Epoch we found variations in F1 scores such as in the Fig. 6, if we consider varying parameter- Learning Rate = 1e-5, Hidden Layers = 768, and Batch = 128, then we found best F1 score 0.95095 at epoch 5 but in terms of Learning Rate = 1e-7, Hidden Layers = 384, and Batch = 32 we found best F1 score 0.94095.

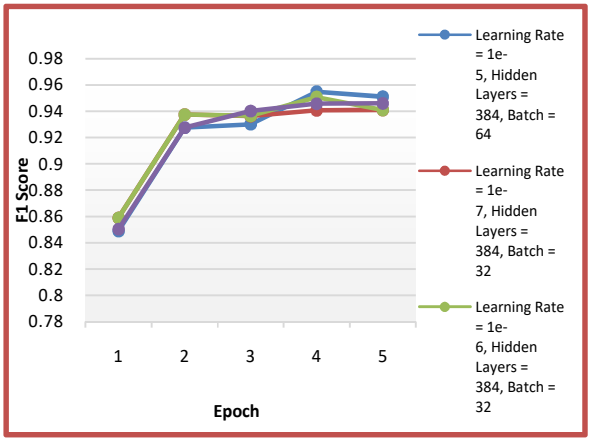


Fig. 6. Performance Evaluation for–Opcode (CNN)

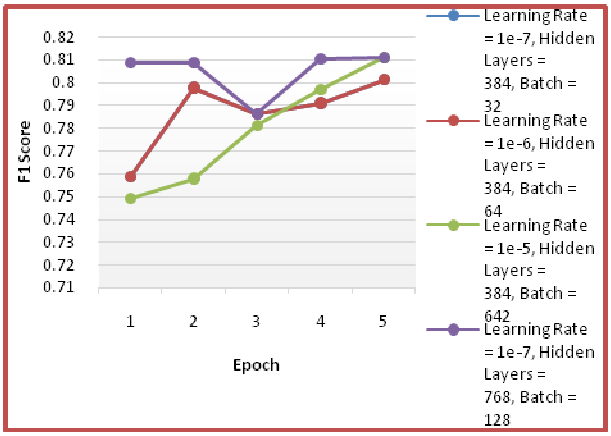


Fig. 7. Performance Evaluation for–API calls (CNN)

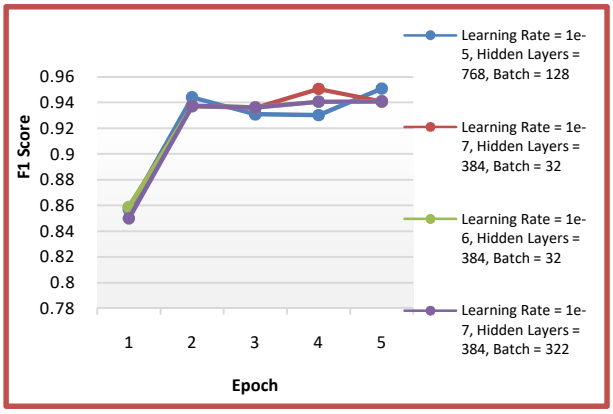


Fig. 8. Performance Evaluation for–Permission (CNN)

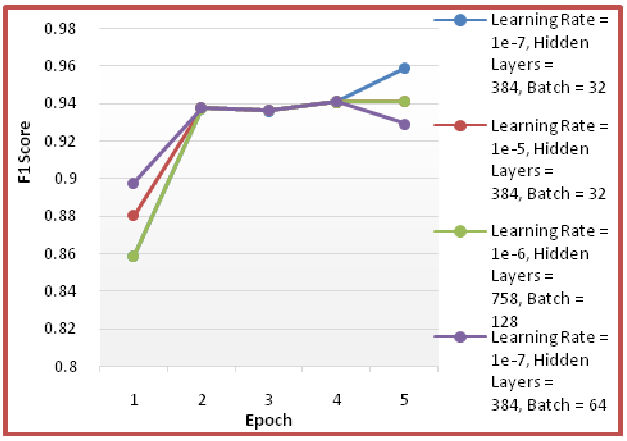


Fig. 9. Performance Evaluation for Multi-featured view: Opcode+API+Permissions(CNN)

In Fig. 7, we can observe that for hyperparameters Learning Rate = 1e-5, Hidden Layers = 768, and Batch = 128, we found best result.

E. *Features Observation*

We extracted seven different features from the Drebin Dataset. By using RF algorithm, we ranked the top three as follows Opcode, Permissions and API are most weighted and have high F1 Scores (Respectively, 95%, 95%, and 81%) compared to the rest of the features. In the Fig. 10, experimental results have been given for all extracted features.

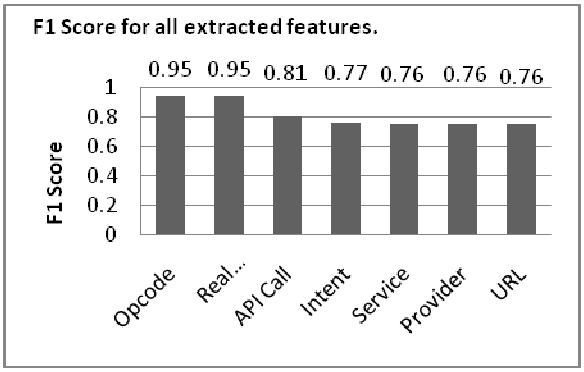


Fig. 10. Performance Evaluation for all the Features

We assume this happened because finding natural writing pattern in intents, provider or websites are not very

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:59:56 UTC from IEEE Xplore. Restrictions apply.

practical and so the F1 score decrease for each feature those are more database oriented not pattern oriented. Hence, we pick Opcode, Permissions and API and feed them to our multi view model for a combined Deep Neural Network.

F. Multi-view Experimental Results – Opcode + Permission + API (CNN)

We next integrate our single feature into a combination of multi-feature set model to observe the variations of performance. Having demonstrated that the input feature sets are useful separately in a single view, it is anticipated that there may be some variation in performance if more than one is employed, demonstrating the complementary nature of the feature sets. In order to evaluate if detection can be enhanced, we have integrated our feature sets in this experiment and we found that performance has been improved which is 96%. So, we can conclude it like that way, if we are considering multiple features together instead of single feature than accuracy level will be improved which can be significant to detect Zero-day malware. In the Fig. 10 experimental results graphical representation has been given and in the Fig. 11, we can notice that combination of multiple features F1 score is better than single feature.

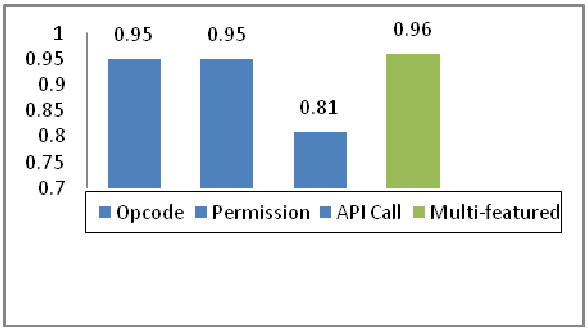


Fig. 11. Single Feature verses Multiple Features Performance

Finally, if we are comparing our findings with the existing works, we can notice that in gap analysis part, we have discussed that there are limitations of features observation and combination but we have considered each one of the features mentioned in Table I and we have developed a multiple featured combination model for preventing zero-day attacks. Also, our developed model performance is pretty good (96%).

V. CONCLUSION

Numerous important conclusions are drawn from our experiments. In our paper, our developed static analysis-based Android malware detection model which is CNN-based neural network (combination of multiple features) that learns from opcodes, permissions, and private Android APIs is able to detect Zero-day Attack scenario. The key take away from this is that this proposed model can reduce overfitting to the training data and generalize better to effectively detect unknown malware. As a consequence, a system is created that is better suited to defend against emerging threats and malware. We are observing that Android Malware and its detection approaches are constantly growing. As a consequence, we must be focused on further analysis how to mitigate these malware issues and obviously about their detection techniques. In terms of machine learning approaches, one of the recent research topics known as AutoML, might be a worthwhile research direction to look into. As of now we are aware of that, the detection techniques and malware are expanding, as a result in future it will be required to give more focus on these research reviews to draw a veil over these expanding threats and their detection methods. In our Deep Learning model, the approach of combinations of three features that better uncovers malware's fundamental characteristics and merges supplemental data to better identify malware in zero-day situations is one of our important accomplishments.

REFERENCES

[1] Statcounter, "Mobile Operating System Market Share Worldwide," 2023. Accessed: Oct. 6, 2023. [Online]. Available: https://gs.statcounter.com/os-market-share/mobile/worldwide

[2] K. Liu, S. Xu, G. Xu, M. Zhang, D. Sun, and H. Liu, "A review of androidmalware detection approaches based on machine learning," *IEEE Access,* Vol. 8, 2020.

[3] S. Garg, and N. Baliyan, "Android security assessment A review, taxonomyand research gap study," *Computers & Security,* Vol. 100, 2021.

[4] J. Khan, and S. Shahzad, "Android Architecture and Related Security Risks," *Asian J. Technol. Manag.,* Vol. 5, pp. 14-18, 2015.

[5] N. Herron, W. B. Glisson, J. T. McDonald, and R. K. Benton, "Machine Learning-based Android Malware Detection Using Manifest Permissions," in *Proceedings of the 54th Hawaii International Conference on System Sciences.*, Jan. 2021.

[6] J. Li, L. Sun, Q. Yan, Z. Li, W. Srisa-an and H. Ye, "Significant Permission Identification for Machine-Learning-Based Android Malware Detection," *IEEE Transactions on Industrial Informatics,* Vol. 14, pp. 3216-3225, 2018.

[7] L. Cai, Y. Li, and Z. Xiong, "JOWMDroid: Android Malware Detection based on Feature Weighting with Joint Optimization of Weight-mapping and Classifier Parameters," *Computers & Security*, Vol. 100, 2021.

[8] L. N. Vu, and S. Jung, "AdMat: A CNN-on-Matrix Approach to Android Malware Detection and Classification," *IEEE Acces,* Vol. 9, pp. 39680-39694, 2021.

[9] A. Mahindru, and A. L. Sangal, "MLDroid—Framework for Android Malware Detection Using Machine Learning Techniques," *Neural Computing and Applications,* Vol. 33(10), pp. 5183-5240, 2021.

[10] A. Daniel, S. Michael, H. Malte, G. Hugo, and R. Konrad, "Drebin: Efficient and Explainable Detection of Android Malware in Your Pocket," in *21th Annual Network and Distributed System Security Symposium (NDSS).,* Feb. 2014.

[11] S. Millar, N. McLaughlin, J. M. Rincon, and P. Miller, "Multi-viewDeepLearning for Zero-day AndroidMalwareDetection," *Journal of Information Securityand Applications,* Vol. 58, 2021.

[12] T. Chen, Q. Mao, Y. Yang, M. Lv, and J. Zhu, "TinyDroid: a lightweight and efficient model for android malware detection and classification," *Mobile information systems*, pp. 1-9, 2018.

[13] F. Maggi, A. Valdi, and S. Zanero, "AndroTotal: A flexible, scalable toolbox and service for testing mobile malware detectors," in *Proceedings ofthe Third ACM workshop on Security and privacy in smartphones & mobile devices.,* Nov. 2013, pp. 49-54.

[14] J. Senanayake, H. Kalutarage, and M. O. Al-Kadri, "Android Mobile Malware Detection Using Machine Learning," *A systematic review of Electronics*, Vol. 10(13), pp. 1606, 2021.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 09,2026 at 10:59:56 UTC from IEEE Xplore. Restrictions apply.